

Question 1 :

1. Big-M Method: Develop a Python program to solve a well-known constrained optimization problem using the Big-M Simplex Method. The selected problem should be formulated as a Linear Programming Problem (LPP), converted into standard form using appropriate slack, surplus, and artificial variables, and solved computationally to obtain the optimal decision-variable values and the corresponding optimal objective-function value. *

```
C:\> mycode > 01_assignment_1_2501ai49 > big_m_method_question_1.py > ...
1  import numpy as np
2
3  M = 1000000
4
5  A = np.array([
6      [1, 1, -1, 0, 1],
7      [1, 2, 0, 1, 0]
8  ], dtype=float)
9
10 b = np.array([4, 6], dtype=float)
11
12 c = np.array([3, 2, 0, 0, -M], dtype=float)
13
14 tableau = np.zeros((3, 7))
15 tableau[:2, :5] = A
16 tableau[:2, 5] = b
17 tableau[2, 5] = -c
18
19 tableau[2] += M * tableau[0]
20
21 basis = [4, 3]
22
23 while True:
24     zj_cj = tableau[2, :5]
25
26     if np.all(zj_cj >= 0):
27         break
28
29     entering = np.argmin(zj_cj)
30
31     ratios = []
32     for i in range(2):
33         if tableau[i, entering] > 0:
34             ratios.append(tableau[i, 5] / tableau[i, entering])
35         else:
36             ratios.append(float('inf'))
37
38     leaving = np.argmin(ratios)
39
40     pivot = tableau[leaving, entering]
41     tableau[leaving] /= pivot
42
43     for i in range(3):
44         if i != leaving:
45             tableau[i] -= tableau[i, entering] * tableau[leaving]
46
47     basis[leaving] = entering
48
49 x = np.zeros(5)
50
51 for i in range(2):
52     x[basis[i]] = tableau[i, 5]
53
54 print("x1 =", x[0])
55 print("x2 =", x[1])
56 print("Z =", tableau[2, 5])
```

Code -

Terminal results -

```

PS C:\Users\yasha\AppData\Local\Programs\Microsoft VS Code> & C:\Python314\python.exe c:/mycode/ot_assignment_1_2501ai49/big_m_method_question_1.py
x1 = 6.0
x2 = 0.0
Z = 18.0
PS C:\Users\yasha\AppData\Local\Programs\Microsoft VS Code>

```

Question 2:

Corresponding optimal objective function value.

2. Transportation Problem: Develop a Python program to solve a well-known transportation optimization problem using (i) **Vogel's Approximation Method (VAM)** and (ii) **MODI (Modified Distribution) Method** (use one method at a time) to obtain an initial basic feasible solution to test for optimality and iteratively improve the allocation. The selected transportation problem should include appropriate source capacities, destination demands, and transportation costs, and the program should determine the optimal shipment plan and the minimum total transportation cost.

Code :

```

import sys

sys.setrecursionlimit(2000)

class TransportationSolver:
    def __init__(self, costs, supply, demand):
        self.original_costs = [row[:] for row in costs]
        self.costs = [row[:] for row in costs]
        self.supply = supply[:]
        self.demand = demand[:]
        self.rows = len(supply)
        self.cols = len(demand)
        self.allocation = [[0] * self.cols for _ in range(self.rows)]

        total_supply = sum(self.supply)
        total_demand = sum(self.demand)

        if total_supply != total_demand:
            print(f"\nSupply: {total_supply}, Demand: {total_demand}")
            self._balance_problem(total_supply, total_demand)

    def _balance_problem(self, total_supply, total_demand):
        if total_supply > total_demand:
            diff = total_supply - total_demand
            self.demand.append(diff)
            for r in range(self.rows):
                self.costs[r].append(0)
            self.cols += 1
        else:
            diff = total_demand - total_supply
            self.supply.append(diff)
            self.costs.append([0] * self.cols)
            self.rows += 1

        self.allocation = [[0] * self.cols for _ in range(self.rows)]

```

```

def solve(self):

    self._run_vam()
    current_cost = self._calculate_total_cost()
    self._print_matrix("initial basic feasible solution", current_cost)

    iteration = 1
    while True:

        is_optimal, deltas, u, v = self._modi_check_optimality()

        if is_optimal:
            print("optimal solution found ")
            break

        min_delta = float('inf')
        enter_i, enter_j = -1, -1

        for i in range(self.rows):
            for j in range(self.cols):

                if self.allocation[i][j] == 0 and deltas[i][j] < min_delta:
                    min_delta = deltas[i][j]
                    enter_i, enter_j = i, j

        print(f"iteration {iteration}: cell ({enter_i+1},{enter_j+1}) enters basis (delta={min_delta}).")

        self._perform_stepping_stone(enter_i, enter_j)

        current_cost = self._calculate_total_cost()
        self._print_matrix(f"after iteration {iteration}", current_cost)
        iteration += 1

    return self.allocation, current_cost

```

```

78 class TransportationSolver:
79     def _run_vam(self):
80
81         temp_supply = self.supply[:]
82         temp_demand = self.demand[:]
83         temp_costs = [row[:]] for row in self.costs
84         alloc = [[0] * self.cols for _ in range(self.rows)]
85
86         while sum(temp_supply) > 0:
87
88             row_penalties = []
89             for r in range(self.rows):
90                 if temp_supply[r] == 0:
91                     row_penalties.append(-1)
92                     continue
93
94                 active_costs = [temp_costs[r][c] for c in range(self.cols) if temp_demand[c] > 0]
95                 if len(active_costs) >= 2:
96                     sorted_costs = sorted(active_costs)
97                     row_penalties.append(sorted_costs[1] - sorted_costs[0])
98                 elif len(active_costs) == 1:
99                     row_penalties.append(active_costs[0])
100                 else:
101                     row_penalties.append(-1)
102
103             col_penalties = []
104             for c in range(self.cols):
105                 if temp_demand[c] == 0:
106                     col_penalties.append(-1)
107                     continue
108                 active_costs = [temp_costs[r][c] for r in range(self.rows) if temp_supply[r] > 0]
109                 if len(active_costs) >= 2:
110                     sorted_costs = sorted(active_costs)
111                     col_penalties.append(sorted_costs[1] - sorted_costs[0])
112                 elif len(active_costs) == 1:
113                     col_penalties.append(active_costs[0])
114                 else:
115                     col_penalties.append(-1)
116
117             max_rp = max(row_penalties) if row_penalties else -1
118             max_cp = max(col_penalties) if col_penalties else -1
119
120             target_r, target_c = -1, -1
121             select_row = True

```

```

if max_rp >= max_cp:
    select_row = True
    target_r = row_penalties.index(max_rp)
else:
    select_row = False
    target_c = col_penalties.index(max_cp)

if select_row:
    min_val = float('inf')
    min_c = -1
    for c in range(self.cols):
        if temp_demand[c] > 0 and temp_costs[target_r][c] < min_val:
            min_val = temp_costs[target_r][c]
            min_c = c

    qty = min(temp_supply[target_r], temp_demand[min_c])
    alloc[target_r][min_c] = qty
    temp_supply[target_r] -= qty
    temp_demand[min_c] -= qty
else:
    min_val = float('inf')
    min_r = -1
    for r in range(self.rows):
        if temp_supply[r] > 0 and temp_costs[r][target_c] < min_val:
            min_val = temp_costs[r][target_c]
            min_r = r

    qty = min(temp_supply[min_r], temp_demand[target_c])
    alloc[min_r][target_c] = qty
    temp_supply[min_r] -= qty
    temp_demand[target_c] -= qty

self.allocation = alloc

```

```

def _get_basic_cells(self):
    basics = []
    for i in range(self.rows):
        for j in range(self.cols):
            if self.allocation[i][j] != 0:
                basics.append((i, j))
    return basics

def _modi_check_optimality(self):
    basics = self._get_basic_cells()
    required_num_basics = self.rows + self.cols - 1

    if len(basics) < required_num_basics:
        self._handle_degeneracy(basics, required_num_basics)
        basics = self._get_basic_cells()

    u = [None] * self.rows
    v = [None] * self.cols
    u[0] = 0

    changed = True
    iterations = 0
    while changed and iterations < 1000:
        changed = False
        iterations += 1
        for (r, c) in basics:
            if u[r] is None and v[c] is not None:
                u[r] = self.costs[r][c] - v[c]
                changed = True
            elif v[c] is None and u[r] is not None:
                v[c] = self.costs[r][c] - u[r]
                changed = True

```

```
def _modify_check_optimality(self):
```

```
    deltas = [[0]*self.cols for _ in range(self.rows)]
    is_optimal = True
    for r in range(self.rows):
        for c in range(self.cols):
            if (r,c) not in basics:
                deltas[r][c] = self.costs[r][c] - (u[r] + v[c])
                if deltas[r][c] < 0:
                    is_optimal = False

    return is_optimal, deltas, u, v
```

```
def _handle_degeneracy(self, current_basics, required):
```

```
    count_needed = required - len(current_basics)
    if count_needed <= 0: return
```

```
    added = 0
    for i in range(self.rows):
        for j in range(self.cols):
            if added >= count_needed: return
            if self.allocation[i][j] == 0:
                self.allocation[i][j] = 0
                added += 1
```

```
def _perform_stepping_stone(self, enter_r, enter_c):
```

```
    path = []
```

```
    found_loop = [False]
    loop_path = []
```

```
def search(r, c, came_from, curr_path):
    if found_loop[0]: return
```

```
    if r == enter_r and c == enter_c and len(curr_path) > 0:
        found_loop[0] = True
        loop_path.extend(curr_path)
        return
```

```

233 def search(r, c, came_from, curr_path):
242     if (r, c) in [(p[0], p[1]) for p in curr_path]: return
243
244
245
246     if came_from == 'ROW' or came_from == 'START':
247
248         for n_r in range(self.rows):
249             if n_r == r: continue
250             if self.allocation[n_r][c] != 0:
251                 search(n_r, c, 'COL', curr_path + [(n_r, c)])
252
253     if came_from == 'COL' or came_from == 'START':
254
255         for n_c in range(self.cols):
256             if n_c == c: continue
257             if self.allocation[r][n_c] != 0:
258                 search(r, n_c, 'ROW', curr_path + [(r, n_c)])
259
260
261     started = False
262     for k in range(self.cols):
263         if k != enter_c and self.allocation[enter_r][k] != 0:
264             search(enter_r, k, 'ROW', [(enter_r, k)])
265             started = True
266             break
267
268     if not found_loop[0]:
269
270         for k in range(self.rows):
271             if k != enter_r and self.allocation[k][enter_c] != 0:
272                 search(k, enter_c, 'COL', [(k, enter_c)])
273                 break
274
275     if not found_loop[0]:
276         print(f"Error: Could not find closed loop for ({enter_r},{enter_c})")
277         return
278
279
280
281     min_qty = float('inf')
282     subtract_indices = []

```

```

    for i in range(len(loop_path)):
        if i % 2 == 0:
            r, c = loop_path[i]
            val = self.allocation[r][c]
            if val < min_qty:
                min_qty = val

    theta = min_qty

    self.allocation[enter_r][enter_c] += theta

    for i in range(len(loop_path)):
        r, c = loop_path[i]
        if i % 2 == 0:
            self.allocation[r][c] -= theta
        else:
            self.allocation[r][c] += theta

    for i in range(self.rows):
        for j in range(self.cols):
            if abs(self.allocation[i][j]) < 1e-9:
                self.allocation[i][j] = 0

def _calculate_total_cost(self):
    total = 0
    for i in range(self.rows):
        for j in range(self.cols):
            total += self.allocation[i][j] * self.costs[i][j]
    return total

```

```

def _print_matrix(self, title, total_cost):
    header = " " * 12 + "".join([f"D{j+1:<10}" for j in range(self.cols)]) + "Supply"
    print(header)
    for i in range(self.rows):
        row_str = f"S{i+1:<12}"
        for j in range(self.cols):
            val = self.allocation[i][j]
            if val > 0:
                row_str += f"{val:<10}"
            else:
                row_str += "-          "
        row_str += str(self.supply[i])
        print(row_str)

    footer = " " * 12 + "".join([f"{self.demand[j]:<10}" for j in range(self.cols)])
    print(footer)
    print(f"Total Cost: {total_cost}")

if __name__ == "__main__":
    costs_data = [
        [3, 1, 7, 4],
        [2, 6, 5, 9],
        [8, 3, 3, 2]
    ]

    supply_data = [300, 400, 500]
    demand_data = [250, 350, 400, 200]

    try:
        solver = TransportationSolver(costs_data, supply_data, demand_data)
        final_alloc, min_cost = solver.solve()

        print(f"minimum cost: {min_cost}")

    except Exception as e:
        print(f"An error occurred: {e}")
        import traceback
        traceback.print_exc()

```

Terminal result:

```

PS C:\Users\yasha\AppData\Local\Programs\Microsoft VS Code> & C:\Python314\python.exe c:/mycode/ot_assignment1.py

```

	D1	D2	D3	D4	Supply
S1	-	300	-	-	300
S2	250	-	150	-	400
S3	-	50	250	200	500
	250	350	400	200	

```

Total Cost: 2850
optimal solution found
minimum cost: 2850
PS C:\Users\yasha\AppData\Local\Programs\Microsoft VS Code>

```